TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

HOTLINE: 18
www.vhu

LẬP TRÌNH JAVA NÂNG CAO

LẬP TRÌNH GIAO DIỆN (GUI) VỚI SWING

ThS. Trương Bá Vĩnh

1

NG ĐẠI HỌC VĂN HIẾN
nh nhân trước thành danh

HOTLINE: 18
www.vhu

Nội dung

- Frame, Panel
- Label, Textfield, Button
- Các đối tượng đồ họa thường dùng khác

2

Khái niệm

- Lập trình giao diện GUI (Graphic User Interface) là việc sử dụng các đối tượng trong Java để thiết kế thành các giao diện trực quan giúp người dùng có thể tương tác để thực hiện các công việc.
- Trong một bài toán, có thể có 1 hoặc nhiều giao diện người dùng.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

Các thư viện lập trình giao diện

- AWT
- SWING

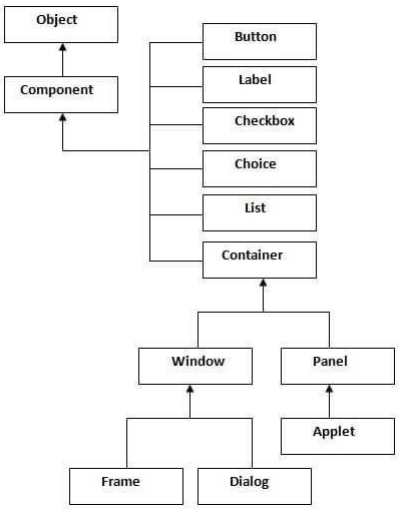
Thư viện lập trình AWT

(Abstract Window Toolkit)

- AWT là bộ thư viện được Java xây dựng từ phiên bản JDK 1.0 để hỗ trợ thiết kế các giao diện và đồ họa người dùng.
- Đây được xem là bộ thư viện cồng kềnh, gặp khó khăn và giao diện không ổn định trên các hệ điều hành khác nhau

AWT

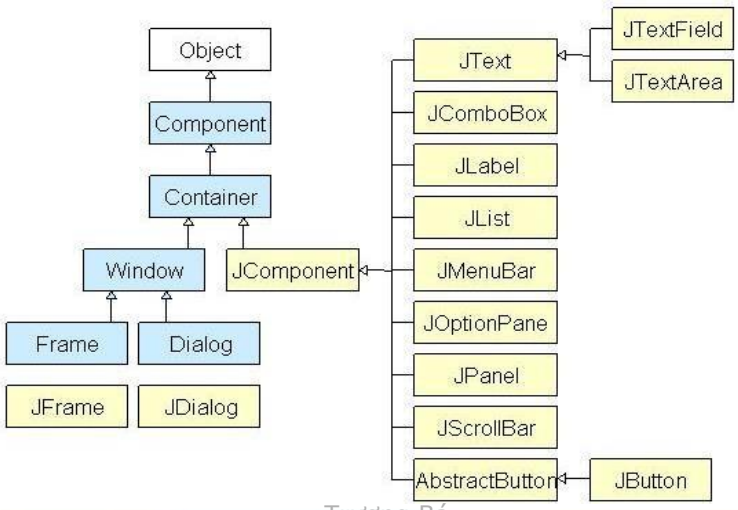
- Kiến trúc một số đối tượng AWT



Thư viện lập trình Swing

- Swing được xây dựng và tích hợp vào phiên bản JDK 1.1.2
- Được đánh giá là bộ thư viện tối ưu và thuận tiện cho việc thiết kế các giao diện người dùng, thân thiện và phù hợp cho nhiều hệ điều hành khác nhau.
- Java Swing là một phần của Java Foundation Classes (JFC) được sử dụng để tạo các ứng dụng Window-Based. Nó được xây dựng ở trên cùng của AWT API và được viết hoàn toàn bằng Java.

Kiến trúc một số đối tượng Swing



Điểm khác nhau giữa AWT và SWING

Java AWT	Java SWING
Các thành phần AWT là phụ thuộc nền tảng	Các thành phần Java Swing là độc lập nền tảng
Các thành phần AWT là nặng	Các thành phần Swing là gọn nhẹ
AWT không hỗ trợ pluggable look & feel	Swing hỗ trợ pluggable look & feel
AWT cung cấp ít thành phần hơn Swing	Swing cung cấp các thành phần mạnh mẽ hơn như table, list, scrollpanes, colorchooser, tabbedpane ...
AWT không theo sau MVC (Model View Controller), ở đây model biểu diễn dữ liệu, view biểu diễn sự trình bày và controller hoạt động như một Interface giữa model và view	Swing theo sau MVC

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

Các thành phần GUI

- Các phần tử UI: : Đó là các phần tử nhìn thấy chủ yếu mà người dùng cuối cùng nhìn thấy và tương tác với. Swing cung cấp rất nhiều các phần tử đa dạng từ cơ bản tới nâng cao.
- Layout: Chúng định nghĩa cách các phần tử UI nên được tổ chức trên màn hình và cung cấp đối tượng L&F (là viết tắt của Look and Feel) cuối cùng tới GUI (Graphical User Interface).
- Hành vi: Đó là các sự kiện xảy ra khi người dùng tương tác với các phần tử UI.

Các phần tử Swing UI

- Lớp JLabelMột đối tượng JLabel là một thành phần để đặt text vào trong một Container
- Lớp JButtonLớp này tạo một button đã được gán nhãn
- Lớp JTableLớp JTable được sử dụng để hiển thị dữ liệu trên các ô của bảng hai chiều
- Lớp GraphicsLớp này cung cấp nhiều phương thức để lập trình đồ họa
- Lớp JColorChooserMột JColorChooser cung cấp một pane gồm các control được thiết kế để cho phép một người dùng thao tác và lựa chọn màu

Các phần tử Swing UI

- Lớp JCheckBoxMột JCheckBox là một thành phần đồ họa mà có thể trong trạng thái on (true) hoặc off (false)
- Lớp JRadioButtonLớp JRadioButton là một thành phần đồ họa mà có thể trong trạng thái on (true) hoặc off (false) trong một nhóm
- Lớp JListMột thành phần JList biểu diễn cho người dùng một danh sách các item
- Lớp JComboBoxMột thành phần JComboBox biểu diễn cho người dùng một menu các lựa chọn

Các phần tử Swing UI

- JTextFieldMột đối tượng JTextField là một thành phần text cho phép chỉnh sửa một dòng text đơn
- Lớp JTextAreaMột đối tượng JTextArea là một thành phần text cho phép sửa đổi một text có nhiều dòng
- Lớp ImageIconMột ImageIcon control là một trình triển khai của Icon Interface mà tô màu các Icon từ Image
- Lớp JScrollbarMột Scrollbar control biểu diễn một thành phần scroll bar để cho người dùng

Các phần tử Swing UI

- Lớp JOptionPaneJOptionPane cung cấp tập hợp các dialog box chuẩn mà gợi ý người dùng về một giá trị hoặc thông báo cho họ một cái gì đó
- JFileChooserMột JFileChooser control biểu diễn một dialog window từ đó người dùng có thể lựa chọn một file
- Lớp JProgressBarThanh tiến trình hiển thị phần trăm hoàn thành tác vụ đang diễn ra
- Lớp JSliderMột JSlider cho phép người dùng lựa chọn một giá trị từ một dãy cụ thể
- Lớp JSpinnerMột JSpinner là một trường input dòng đơn, cho phép người dùng lựa chọn một số hoặc một giá trị đối tượng từ dãy đã qua sắp xếp

Các đối tượng Container trong Java

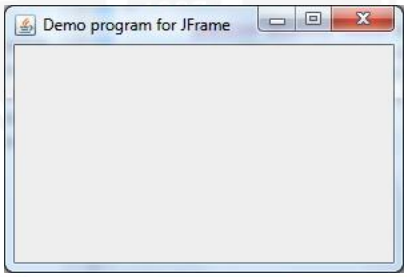
- **Container** là vùng để đặt các thành phần giao diện vào đó. Bất cứ vật gì mà kế thừa từ lớp Container sẽ là vật chứa.
- Một vật chứa có thể chứa nhiều phần tử, các phần tử này có thể được vẽ hay được tô màu tuỳ thích. Bạn hãy xem vật chứa như một cửa sổ.
- Một số container thường gặp:
  - Frames
  - Panels
  - Dialogs
  - …

Một số phương thức thường dùng của Java Swing trong Component class

- public void add(Component c) : thêm 1 component trên 1 component khác
- public void setSize(int width, int height): thiết lập kích thước Component
  - Ví dụ: setSize(300,100);
- public void setVisible(Boolean b): thiết lập thuộc tính ẩn hay hiện cho Component (giá trị ngầm định là false).
- public void setLayout(layoutManager m): thiết lập layout chính cho **Container**

JFrame

- JFrame trong gói Swing kế thừa từ Frame của AWT. JFrame giống như cửa sổ chính có thể chứa trong nó các thành phần để tạo Gui như: labels, buttons, textfields, …
- Ưu điểm của JFrame hơn Frame: nó có thêm tùy chọn hide và close cửa sổ thông qua phương thức *setDefaultCloseOperation(int)*.



TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

- Code tạo JFrame có thể được viết trong các constructor.
  - JFrame(): Tạo một Frame mới, ban đầu là không nhìn thấy (invisible)
  - JFrame(String title): Tạo một Frame mới, ban đầu là không nhìn thấy (invisible) với title đã cho.
- Chúng ta cũng có thể kế thừa lớp JFrame, nên không cần tạo thể hiện của các JFrame

JFrame – Một số phương thức thường dùng

- setTitle(String Title): định nghĩa tiêu đề cho khung giao diện
  - Ví dụ: setTitle("My first program");
- setSize(int width, int height): định nghĩa kích thước chiều rộng và chiều cao của khung giao diện.
  - Ví dụ: setSize(300,100);
- setBackground(color c); định nghĩa màu sắc cho nền của Jframe
  - Ví dụ: getContentPane().setBackground(Color.RED);

JFrame – Một số phương thức thường dùng

- setLocation(int x, int y): định nghĩa vị trí hiển thị của khung giao diện trong màn hình.
  - Ví dụ: setLocation(20,20);
- setBounds(int x, int y, int width, int height): định nghĩa vị trí và kích thước cho khung giao diện.
  - Ví dụ: setBounds(20,20,300,100);
- setResizable(Boolean value); value=true cho phép người dùng có thể kéo dãn kích thước khung giao diện, ngược lại thì kích thước khung giao diện bị cố định.
  - Ví dụ: setResizable(true);

JFrame – Một số phương thức thường dùng

- setDefaultCloseOperation(int mode); định nghĩa hành động của khung giao diện khi người dùng click vào dấu X ở góc phải trên cùng.
  - DISPOSE_ON_CLOSE: đóng giao diện đang thao tác mà không ảnh hưởng tới các giao diện hiển thị khác. Nếu chương trình chỉ có 1 giao diện thì chương trình sẽ tự động kết thúc khi giao diện được đóng.
  - HIDE_ON_CLOSE: ẩn giao diện đang sử dụng xuống, giống như khi ta click vào – thu nhỏ màn hình. Nó không làm tắt giao diện và các trạng thái trên giao diện vẫn giữ nguyên. **(chế độ mặc định)**

- DO_NOTHING_ON_CLOSE: không thực hiện hành động gì. Trong trường hợp này, thông thường chúng ta sẽ xử lý bắt thao tác của người dung và tự định nghĩa hành vi cho khung giao diện.
- EXIT_ON_CLOSE: đóng giao diện đang thao tác, đồng thời tắt luôn chương trình. Thường được sử dụng cho giao diện chính của chương trình.
- Ví dụ:

```
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

- setLayout(LayoutManager layout); định nghĩa việc hiển thị các đối tượng trong khung giao diện. Layout mặc định của Frame là BorderLayout.
  - null: cho phép các đối tượng trong khung giao diện được hiển thị ở các vị trí tùy ý, phụ thuộc vào việc định nghĩa x, y, width, height của các đối tượng.
  - Ví dụ: setLayout(null);
    setLayout(new CardLayout());

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

JFrame – Một số phương thức thường dùng

- setVisible(Boolean true): định nghĩa việc ẩn/hiện của khung giao diện. Nếu true: khung giao diện hiển thị ngược lại thì bị ẩn.
  - Ví dụ: setVisible(true);
- setLocationRelativeTo(Component comp); định nghĩa việc hiển thị của một đối tượng khung giao diện trong màn hình chứa nó, giúp khung giao diện luôn hiển thị chính giữa.
  - Ví dụ: setLocationRelativeTo(null);
- pack(): hiển thị Frame ôm sát nội dung trên form

JFrame – Một số phương thức thường dùng

- setIconImage(Image icon); định nghĩa icon của khung giao diện.
  - Ví dụ:
  - Khởi tạo đối tượng Image:

Image icon=new ImageIcon (Ten_doi_tuong.class.getResource(String path).getImage();

- Set Icon cho khung giao diện:

setIconImage(Image icon);

JFrame – Một số phương thức thường dùng

- addWindowListener(WindowListener lis); là bộ sự kiện giúp khung giao diện lắng nghe được khi trạng thái của nó bị thay đổi như: bị đóng lại, bị mở ra, bị ẩn, …
- addMouseListener(MouseListener m); là bộ sự kiện giúp khung giao diện lăng nghe khi người dùng sử dụng chuột để tương tác: click, press, move, di chuột vào khung giao diện, …
- addKeyListener(KeyListener k); là bộ sự kiện giúp khung giao diện người dung lắng nghe được khi người dung nhấn các phím cứng trên bàn phím để tương tác với nó.

JFrame – Một số phương thức thường dùng

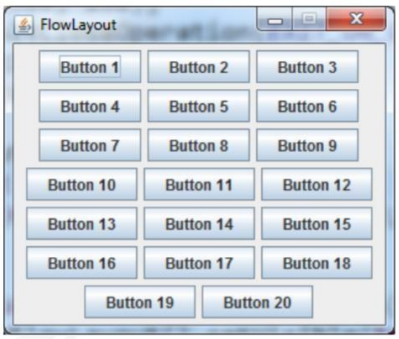
- WindowEvent: là đối tượng chứa tất cả các thông số cần thiết của khung giao diện tương ứng với trạng thái lắng nghe được. Có thể sử dụng đối tượng này để lấy các dữ liệu liên quan của khung giao diện tại thời điểm lắng nghe được.
- MouseEvent: là đối tượng chứa các thông số cần thiết về chuột như (x, y, …) tương ứng với trạng thái khung giao diện đang lắng nghe được.
- KeyEvent: đối tượng chứa các thông tin về phím mà người dùng vừa thao tác trong các trạng thái lắng nghe được, chúng ta có thể thông qua đối tượng này để lấy các thông tin như mã, nội dung phím vừa nhấn là gì.
- add(Component com); phương thức dùng để add một đối tượng giao diện vào khung chứa.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

Layout

FlowLayout

- Đây là loại layout đơn giản nhất, layout này thường được dùng kết hợp với các layout khác.
- Layout này quy định kích thước của các component con là vừa đủ với nội dung hiển thị của component.
- Mặc định các component sẽ được sắp xếp trên một hàng từ trái sang phải, nếu không vừa đủ một hàng thì chúng sẽ xuống hàng.



Layout

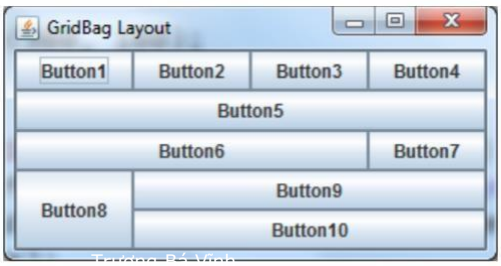
GridLayout

- Layout này sắp xếp các component theo dạng bảng.
- Các component sẽ có kích thước bằng nhau.
- GridLayout(int rows, int cols)
- GridLayout(int rows, int cols, int hgap, int vgap)

Layout

GridBagLayout

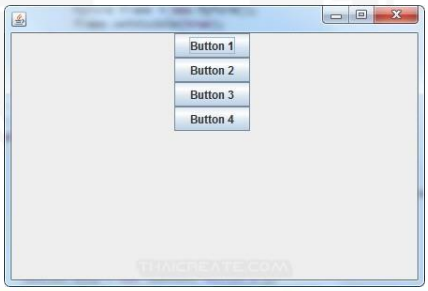
- Đây là một layout có cơ chế sắp xếp giống như GridLayout, các phần tử xếp thành một lưới hình chữ nhật. Tuy nhiên nó linh động hơn GridLayout cho phép chúng ta chỉ định vị trí, kích thước của các phần tử trong lưới hình chữ nhật.
- Mỗi thành phần con trong GridBagLayout đều gắn liền với một GridBagConstraints object chỉ định các thuộc tính như toạ độ dòng và cột, độ rộng của một thành phần con.



Layout

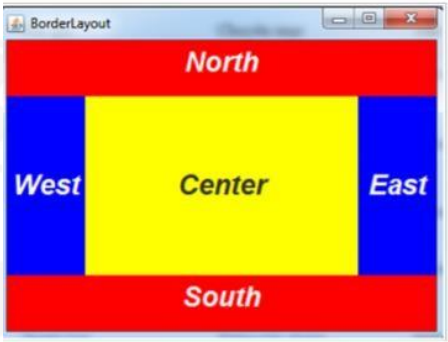
BoxLayout

- BoxLayout cho phép tạo các giao diện phức tạp.
- Sắp xếp các component theo chiều ngang hoặc chiều dọc, khác với FlowLayout, các control sẽ không tự động xuống dòng nếu như không đủ chỗ cho chúng.
- Có thể lồng một BoxLayout vào một BoxLayout khác.



BorderLayout

- Sắp xếp các component theo vùng, ở đây có 5 vùng là Đông (EAST), Tây (WEST), Nam (SOUTH), Bắc (NORTH), và Chính giữa (CENTER).
- Mỗi vùng chỉ được chứa một component, muốn đưa nhiều component vào một vùng thì đặt một layout khác vào vùng đó rồi đặt các component vào layout mới đó.

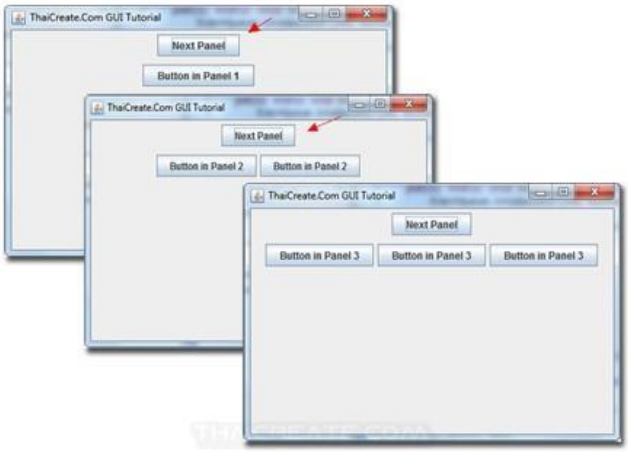


VD: Thêm button b vào frame f tại vị tr
WEST
f.add(b,BorderLayout.WEST);

- Kích thước của các component trong mỗi vùng là do chúng ta thiết lập chứ không phải do layout tự co dãn, ngoại trừ vùng CENTER sẽ có kích thước thay đổi tùy thuộc vào 4 vùng còn lại.

CardLayout

Cho phép các đối tượng giao diện được hiển thị thành các tầng layer, mỗi đối tượng là một tầng. Đối tượng được thêm vào đầu tiên sẽ ở trên cùng



Cách xây dựng đối tượng JFrame

Ví dụ 3.1 File đặt trong tệp: FirstSwingExample.java

```
import javax.swing.*;
public class FirstSwingExample {
public static void main(String[] args) {
JFrame f=new JFrame();//creating instance of JFrame
JButton b=new JButton("click");//creating instance of JButton
b.setBounds(80,80,100, 40);//x axis, y axis, width, height
f.add(b);//adding button in JFrame
f.setSize(300,200);//300 width and 200 height
f.setLayout(null);//using no layout managers
f.setVisible(true);//making the frame visible
}
}
```

import thư viện swing

- Kết quả

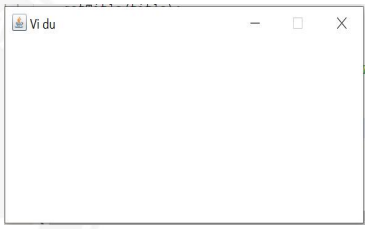
Cách xây dựng đối tượng JFrame

- Ví dụ 3.2:

```
import javax.swing.*;
import java.awt.*;
public class GUI extends JFrame{
    public static final int WIDTH =400;
    public static final int HEIGHT =200;
    public GUI(String title){
        initGUI(title); // gọi phương thức định nghĩa khung giao diện
    }
    private void initGUI(String title){
        setTitle(title);
        setSize(WIDTH,HEIGHT);
        setLocationRelativeTo(null);
```

import thư viện awt và swing

```
        getContentPane().setBackground(Color.WHITE);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        setLayout(new FlowLayout());
        }
    }
    public class Main{
        public static void main(String[] args){
            GUI gui = new GUI("Vi du");
            gui.setVisible(true);
        }
    }
```



JPanel

- JPanel là một lớp container đơn giản nhất. Nó cung cấp không gian để các ứng dụng có thể thêm các component vào. Nó được kế thừa từ lớp JComponent.
- JPanel không có thanh tiêu đề (title bar).

JPanel – Một số constructor phổ biến

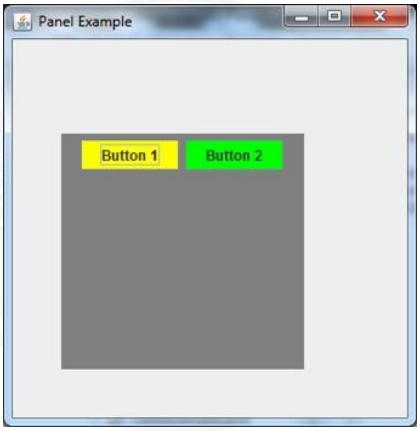
- JPanel(): Tạo một JPanel mới mặc định sử dụng FlowLayout.
- JPanel(LayoutManager layout): Tạo một JPanel mới với Layout Manager đã cho

JPanel – Ví dụ 1

Ví dụ 3.3

```
import java.awt.*;
import javax.swing.*;
public class PanelExample {
    PanelExample()
      {
      JFrame f= new JFrame("Panel Example");
      JPanel panel=new JPanel();
      panel.setBounds(40,80,200,200);
      panel.setBackground(Color.gray);
      JButton b1=new JButton("Button 1");
      b1.setBounds(50,100,80,30);
```

```
b1.setBackground(Color.yellow);
JButton b2=new JButton("Button 2");
b2.setBounds(100,100,80,30);
b2.setBackground(Color.green);
panel.add(b1); panel.add(b2);
f.add(panel);
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);          }
public static void main(String args[])  {
new PanelExample();
}     }
```

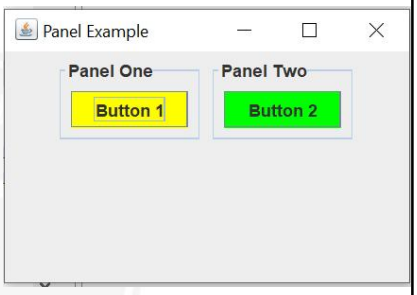



JPanel – Ví dụ 2

```
PanelExample()  {
    JFrame f= new JFrame("Panel Example");
    JPanel panel1=new JPanel();
    JPanel panel2=new JPanel();
    panel1.setSize(100,100);
    panel2.setSize(100,100);
    JButton b1=new JButton("Button 1");
    b1.setBackground(Color.yellow);
    JButton b2=new JButton("Button 2");
    b2.setBackground(Color.green);
```



```
    panel1.add(b1);
    panel2.add(b2);
    f.setLayout(new FlowLayout());
    f.add(panel1);
    f.add(panel2);
    f.setSize(300,200);
    f.setVisible(true);
    panel1.setBorder(BorderFactory.createTitledBorder("Panel One"));
    panel2.setBorder(BorderFactory.createTitledBorder("Panel Two"));
    }
```



JLabel

- JLabel: là đối tượng giao diện hiển thị một tiêu đề có tác dụng chú thích hoặc thông báo trên giao diện
- JLabel: không sửa được văn bản hiển thị đối với người dung.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

JLabel – Một số constructor phổ biến

- JLabel(): tạo một thể hiện JLabel không có hình ảnh và tiêu đề trống.
- JLabel(String s): tạo một thể hiện JLabel với text cụ thể.
- JLabel(Icon i): tạo một thể hiện JLabel với 1 ảnh xác định.
- JLabel(String s, Icon i, int horizontalAlignment): tạo một thể hiện JLabel với text xác định có ảnh và được căn chỉnh theo chiều ngang.

JLabel – Một số phương thức thường dùng

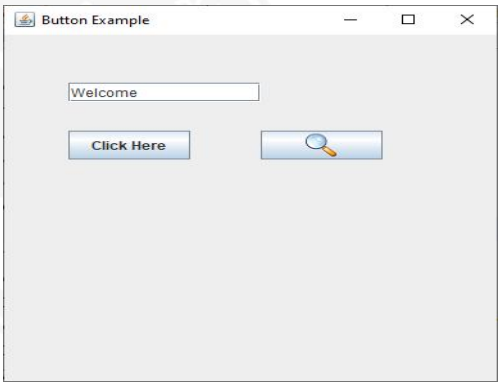
- String getText(): trả về chuỗi mà label đang hiển thị.
- void setText(String text): thiết lập 1 dòng đơn chuỗi text hiển thị trên component.
- void setHorizontalAlignment(int alignment): thiết lập nội dung hiển thị của label theo hang ngang trục X.
- Icon getIcon(): trả về hình ảnh mà label hiển thị.
- int getHorizontalAlignment(): trả về dạng căn chỉnh của nội dung label theo trục X.

JLabel – Ví dụ 3.5

```
import javax.swing.*;
class LabelExample{
public static void main(String args[])    {
   JFrame f= new JFrame("Label Example");
   JLabel l1,l2;
   l1=new JLabel("First Label.");
   l1.setBounds(50,50, 100,30);
   l2=new JLabel("Second Label.");
   l2.setBounds(50,100, 100,30);
   f.add(l1); f.add(l2);
   f.setSize(300,300);
   f.setLayout(null);
   f.setVisible(true);    }    }
```

JTextField

- JTextField: đối tượng dùng để nhập dòng dữ liệu đơn.



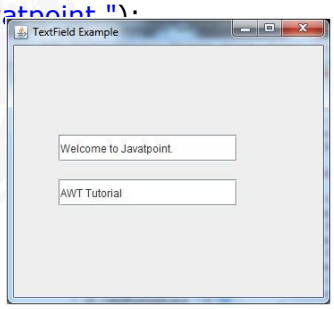
TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

JTextField – Một số constructor thường dùng

- JTextField(): tạo 1 textfield trống mới
- JTextField(String text): tạo 1 textfield có nội dung là text.

JTextField – Ví dụ

```
import javax.swing.*;
class TextFieldExample  {
public static void main(String args[])      {
    JFrame f= new JFrame("TextField Example");
    JTextField t1,t2;
    t1=new JTextField("Welcome to Javatpoint.");
    t1.setBounds(50,100, 200,30);
    t2=new JTextField("AWT Tutorial");
    t2.setBounds(50,150, 200,30);
    f.add(t1); f.add(t2);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);     }     }
```



JButton

- JButton: là một component được sử dụng để tạo ra một nút có nhãn được thực thi một cách độc lập. Thực thi 1 hành động nào đó khi người dung nhấn nút.

JButton – Một số constructor phổ biến

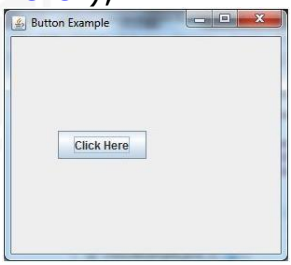
- JButton(): tạo ra 1 nút không có text cũng như ảnh.
- JButton(String s): tạo ra nút có text.
- JButton(Icon i): tạo ra nút có ảnh

JButton – Một số method thường dùng

- void setText(String s): dung để thiết lập text cho nút .
- String getText(): trả về text của nút
- void setEnabled(boolean b): để thiết lập nút được kích hoạt hay không.
- void setIcon(Icon b): thiết lập ảnh cho nút.
- Icon getIcon(): trả về icon đang dung của nút.
- void setMnemonic(int a): thiết lập mnemonic (phím tắt) cho nút.
- void addActionListener(ActionListener a): dùng để thêm action listener cho nút.

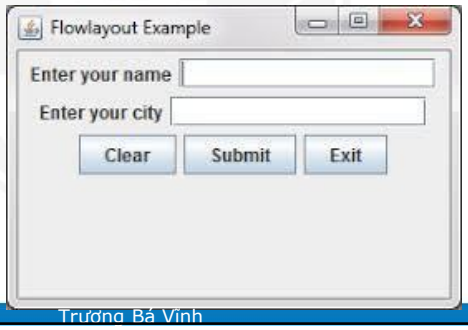
JButton – Ví dụ

```
import javax.swing.*;
public class ButtonExample {
public static void main(String[] args) {
    JFrame f=new JFrame("Button Example");
    JButton b=new JButton("Click Here");
    b.setBounds(50,100,95,30);
    f.add(b);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
} }
```



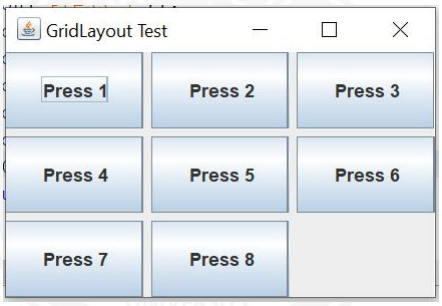
Bài tập

1. Tạo 1 frame 250×200 với tiêu đề “HelloWorld” hiển thị ở tọa độ (300, 200) trên màn hình. Trong Frame có nút click. Tắt chương trình khi người dùng click vào dấu X ở góc phải trên cùng.
2. Thiết kế giao diện sau (sử dụng FlowLayout).

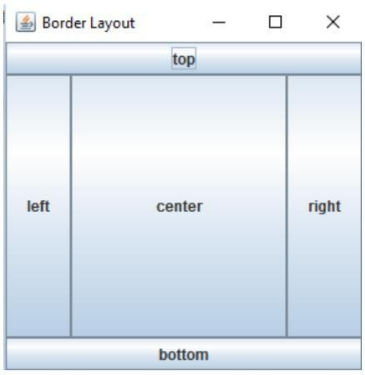


Bài tập

3. Thiết kế giao diện sau (sử dụng GridLayout)

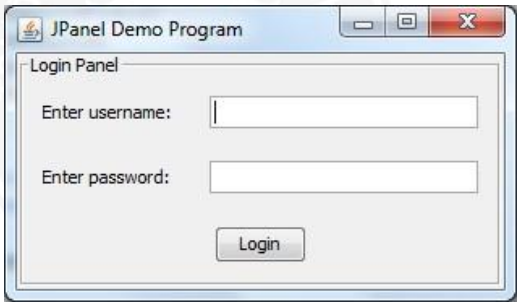


4. Thiết kế giao diện sau (sử dụng BorderLayout)



Bài tập

5. Thiết kế giao diện sau, kết thúc chương trình khi tắt cửa sổ.



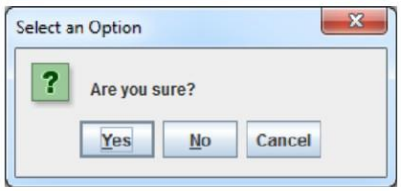
Hiển thị Dialog (JOptionPane)

- Message Dialog

showMessageDialog

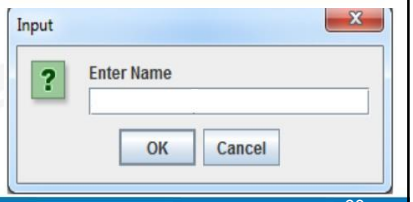
- Confirm Dialog

showConfirmDialog

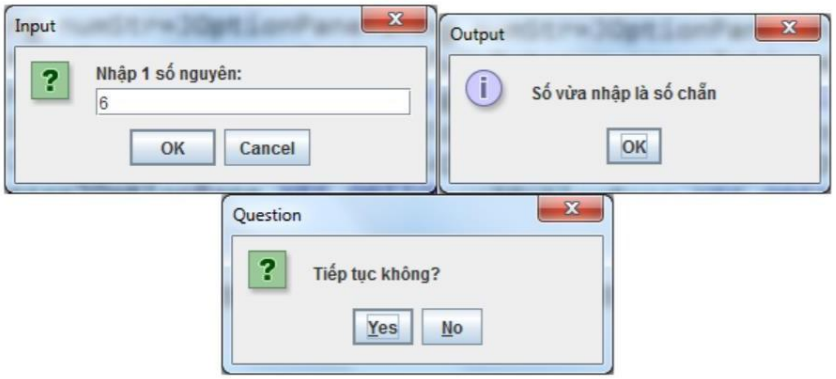


- Input Dialog

showInputDialog



Ví dụ



TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

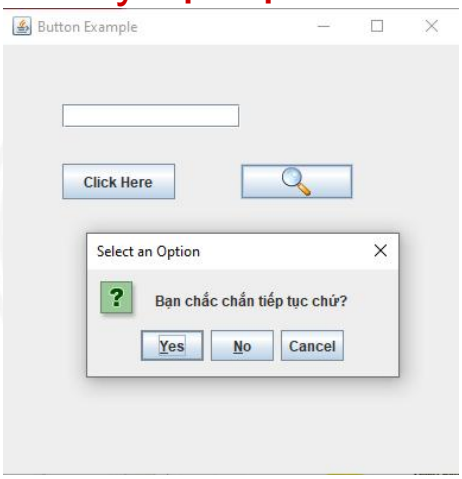
Ví dụ

```
import javax.swing.JOptionPane;
public class EvenOdd {
    public static void main(String[] args) {
        int qes;
        do {
            String numStr=JOptionPane.showInputDialog("Nhập 1 số
nguyên: ");
            if(numStr!=null && !numStr.trim().equals("")) {
                try {
                    int num=Integer.parseInt(numStr);
                    String result="Số vừa nhập là số "+ ((num%2==0)
? "chẵn" : "lẻ");
                    JOptionPane.showMessageDialog(null, result,
"Output",JOptionPane.INFORMATION_MESSAGE); }
```

Ví dụ

```
                catch (NumberFormatException ex){
                    JOptionPane.showMessageDialog(null, "Nhập sai
dữ  liệu");
                }
            }
            qes=JOptionPane.showConfirmDialog(null, "Tiếp tục
không?", "Question",JOptionPane.YES_NO_OPTION);
        } while(qes==JOptionPane.YES_OPTION);
    }
}
```


JButton – Xử lý sự kiện



JButton – Xử lý sự kiện (1)

```
import javax.swing.*; import
java.awt.event.*; public class
JButtonAction {
   public static void main(String[] args) { JFrame f=new
   JFrame("Button Example"); JTextField tf=new
   JTextField(); tf.setBounds(50,50, 150,20);
   JButton b=new JButton("Click Here");
   b.setBounds(50,100,95,30); b.addActionListener(new
   ActionListener(){ @Override
   public void actionPerformed(ActionEvent e){
         tf.setText("Welcome to Javapoint.");
      }    });
```


JButton – Xử lý sự kiện (1)

```
JButton c= new JButton(new ImageIcon("E:\\ImageIcon\\Find.png"));
c.setBounds(200,100 , 95, 30);
c.addActionListener(new ActionListener(){
   @Override
   public void actionPerformed(ActionEvent e){
    JOptionPane.showConfirmDialog(f, "Bạn chắc chắn tiếp tục
chứ?");
   }    });
f.add(b);f.add(tf); f.add(c);
f.setSize(400,400);
f.setLayout(null);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setVisible(true);
}
```

JButton – Xử lý sự kiện (2)

```
import javax.swing.*;
import java.awt.event.*;
public class JButtonAction implements ActionListener {
    JFrame f;
    JButton b,c;
    JTextField tf;
    public JButtonAction() {
    f=new JFrame("Button Example");
    tf=new JTextField();
    tf.setBounds(50,50, 150,20);
    b=new JButton("Click Here");
    b.setBounds(50,100,95,30);
    b.addActionListener(this);
```


JButton – Xử lý sự kiện (2)

```
c= new JButton(new ImageIcon("E:\\ImageIcon\\Find.png"));
c.setBounds(200,100 , 95, 30);
c.addActionListener(this);
f.add(b);f.add(tf); f.add(c);
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

JButton – Xử lý sự kiện (2)

```
@Override
public void actionPerformed(ActionEvent e){
    if (e.getSource() == b) {
        tf.setText("Welcome to Javapoint.");
    } else if (e.getSource() == c) {
        JOptionPane.showConfirmDialog(f, "Bạn chắc chắn tiếp tục chứ?");
    }
  }
  public static void main(String[] args) {
    new JButtonAction();
  }
}
```


TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

JCheckBox

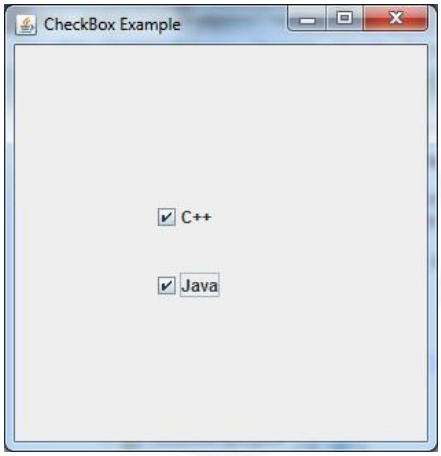
- Đối tượng cho phép chọn/ không chọn giá trị
- Cho phép lựa chọn đồng thời nhiều giá trị
- Một số constructor thường dùng:
  - JCheckBox(): tạo ra 1 nút check box chưa được lựa chọn không có text và icon.
  - JCheckBox(String s): tạo ra 1 checkbox chưa có lựa chọn có text.
  - JCheckBox(String text, boolean selected):tạo 1 checkbox có text, lựa chọn hoặc không được lựa chọn ngay từ đầu.

JCheckBox

- Các thuộc tính thường dùng:
  - Text: văn bản hiển thị trên đối tượng
  - Selected: đối tượng được lựa chọn hay không
- Một số phương thức thường dùng:
  - protected String paramString(): trả về xâu đại diện của JCheckBox.
  - boolean isSelected(): trả về true khi JCheckBox đang được select, false là ngược lại.
  - setSelected(Boolean value): dùng để set trạng thái cho JCheckBox là đang được chọn hoặc đang được bỏ chọn.

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vh

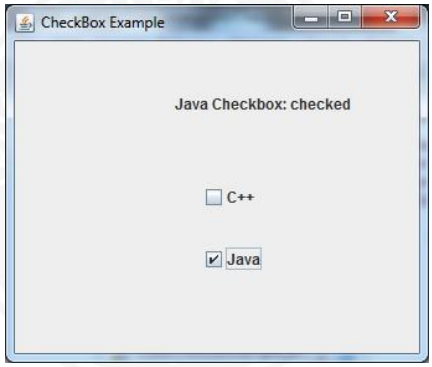
Ví dụ



Ví dụ

```
import javax.swing.*;
public class CheckBoxExample  {
    CheckBoxExample(){
      JFrame f= new JFrame("CheckBox Example");
      JCheckBox checkBox1 = new JCheckBox("C++");
      checkBox1.setBounds(100,100, 50,50);
      JCheckBox checkBox2 = new JCheckBox("Java", true);
    checkBox2.setBounds(100,150, 50,50);
      f.add(checkBox1);     f.add(checkBox2);
      f.setSize(400,400); f.setLayout(null); f.setVisible(true);
   }
public static void main(String args[])      {
   new CheckBoxExample();      }}
```


Ví dụ JCheckBox



Java JCheckBox Example with ItemListener

```
import javax.swing.*;
import java.awt.event.*;
public class CheckBoxExample  {
    CheckBoxExample(){
      JFrame f= new JFrame("CheckBox Example");
      JLabel label = new JLabel();
      label.setHorizontalAlignment(JLabel.CENTER);
      label.setSize(400,100);
      JCheckBox checkbox1 = new JCheckBox("C++");
      checkbox1.setBounds(150,100, 50,50);
      JCheckBox checkbox2 = new JCheckBox("Java");
      checkbox2.setBounds(150,150, 50,50);
      f.add(checkbox1); f.add(checkbox2); f.add(label);
```



```
      checkbox1.addItemListener(new ItemListener() {
          @Override
          public void itemStateChanged(ItemEvent e) {
           label.setText("C++ Checkbox: "
         + (e.getStateChange()==1?"checked":"unchecked"));
          }
        });
      checkbox2.addItemListener(new ItemListener() {
          @Override
          public void itemStateChanged(ItemEvent e) {
           label.setText("Java Checkbox: "
          +
          (e.getStateChange()==1?"checked":"unchecked"));
          }
        });
```

```
f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
public static void main(String args[])
{
    new CheckBoxExample();
}
}
```

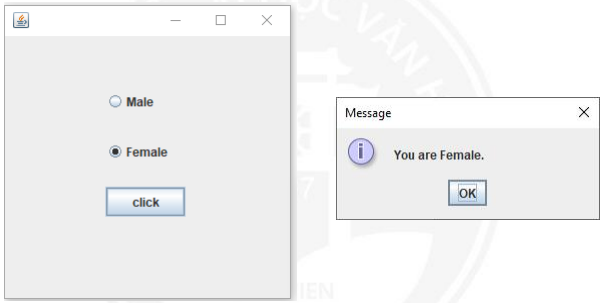

JRadioButton

- Đối tượng cho phép chọn/không chọn giá trị.
- Cho phép chọn một giá trị tại 1 thời điểm
- Một số constructor thường dùng:
  - JRadioButton(): tạo ra 1 radio button không có text và không được lựa chọn.
  - JRadioButton(String s): tạo 1 radio button có text và không được lựa chọn.
  - JRadioButton(String s, boolean selected): tạo radio button có text và trạng thái được chọn.

JRadioButton

- Một số phương thức thường dùng:
  - isSelected(): trả về true nếu đối tượng được check
- Lưu ý:
  - Các radio button nên được đặt trong 1 button group

JRadioButton – Ví dụ 1



JRadioButton – Ví dụ 1

```
public class JRadioEx1 extends JFrame implements ActionListener{
    JRadioButton rb1,rb2;
    JButton b;
    JRadioEx1 (){
        rb1=new JRadioButton("Male");
        rb1.setBounds(100,50,100,30);
        rb2=new JRadioButton("Female");
        rb2.setBounds(100,100,100,30);
        ButtonGroup bg=new ButtonGroup();
        bg.add(rb1);bg.add(rb2);
```

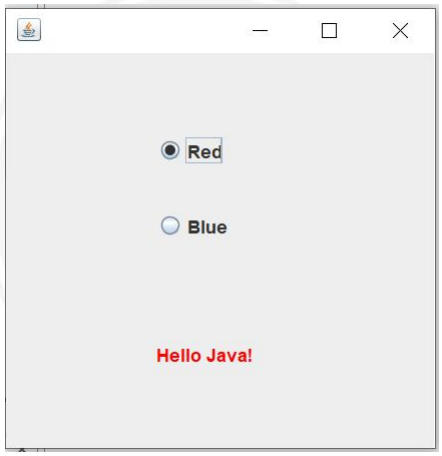

JRadioButton – Ví dụ 1

```
        b=new JButton("click");
        b.setBounds(100,150,80,30);
        b.addActionListener(this);
        add(rb1);add(rb2);add(b);
        setSize(300,300);
        setLayout(null);
        setVisible(true);
    }
```

JRadioButton – Ví dụ 1

```
public void actionPerformed(ActionEvent e){
    if(rb1.isSelected()){
    JOptionPane.showMessageDialog(this,"You are Male.");
    }
    if(rb2.isSelected()){
    JOptionPane.showMessageDialog(this,"You are Female.");
    }
}
public static void main(String args[]){
    new JRadioEx1(); }
}
```


JRadioButton – Ví dụ 2



JRadioButton – Ví dụ 2

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class JRadioEx2 extends JFrame implements ActionListener{
    JRadioButton rb1,rb2;
    JButton b;  JLabel label;
    JRadioEx2 (){
        label = new JLabel("Hello Java!");
        rb1=new JRadioButton("Red");
        rb1.setBounds(100,50,100,30);
        rb2=new JRadioButton("Blue");
        rb2.setBounds(100,100,100,30);
```


JRadioButton – Ví dụ 2

```
        label.setBounds(100,150,100,100);
        ButtonGroup bg=new ButtonGroup();
        bg.add(rb1);bg.add(rb2);
        rb1.addActionListener(this);
        rb2.addActionListener(this);
        add(rb1);add(rb2);add(label);
        setSize(300,300);
        setLayout(null);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
```

JRadioButton – Ví dụ 2

```
public void actionPerformed(ActionEvent e){
    if(rb1.isSelected()){
    label.setForeground(Color.red);
    }
    if(rb2.isSelected()){
    label.setForeground(Color.blue);
    }
}
public static void main(String args[]){
    new JRadioEx2(); }
}
```


JComboBox

- Được dùng để tạo ra các menu trải xuống các lựa chọn. Mục đã chọn sẽ được hiển thị ngay trên đầu của menu.
- Các constructor thường dùng:
  - JComboBox(): tạo 1 combobox với mô hình dữ liệu ngầm định.
  - JComboBox(Object[] items): tạo 1 combobox chứa các phần tử trong mảng chỉ định.
  - JComboBox(Vector<?> items): tạo 1 combobox chứa các phần tử trong vector đã được chỉ định.

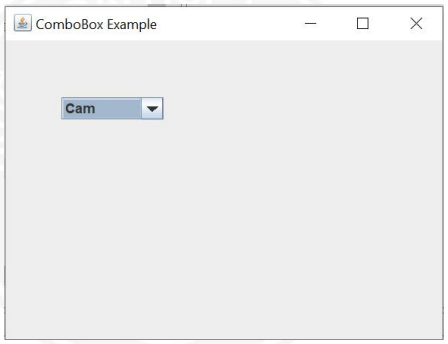
JComboBox- Các phương thức thường dùng

- **getItemAt(int index):** trả về item ở vị trí index, phần tử đầu tiên của combo được hiểu ở vị trí 0, nếu index nằm ngoài phạm vi của combo sẽ trả về null.
- **void addItem(Object anObject):** dùng để thêm 1 item vào danh sách.
- **void removeItem(Object anObject):** dùng để xóa 1 item trong danh sách item.
- **void removeAllItems():** dùng để xóa toàn bộ item trong danh sách.

JComboBox- Các phương thức thường dùng

- **getSelectedIndex():** trả về chỉ số của item được chọn
- **getSelectedItem():** trả về nội dung item đang được lựa chọn
- **void setEditable(boolean b):** dùng để thiết lập Combobox có sửa được hay không.

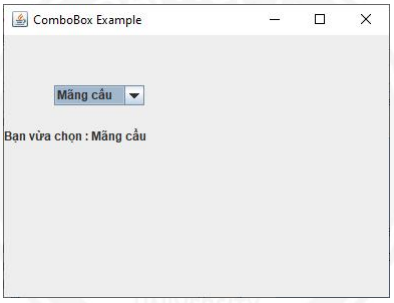
JComboBox – Ví dụ 1



JComboBox – Ví dụ 1

```
import javax.swing.*;
public class JComboExample {
    JFrame f;
    JComboExample(){
        f=new JFrame("ComboBox Example");
        String combo[]={"Cam","Quýt","Ổi","Mãng cầu","Lê"};
        JComboBox cb=new JComboBox(combo);
        cb.setBounds(50, 50,90,20);
        f.add(cb);          f.setLayout(null);
        f.setSize(400,300);     f.setVisible(true);     }
    public static void main(String[] args) {
        new JComboExample();     } }
```

JComboBox – Ví dụ 2



JComboBox – Ví dụ 2

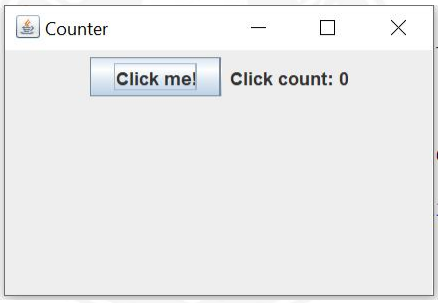

```
public class JComboExample {
    JFrame f;
    JComboExample(){
        f=new JFrame("ComboBox Example");
        String combo[]={"Cam","Quýt","Ổi","Măng cầu","Lê"};
        JComboBox cb=new JComboBox(combo);
        cb.setBounds(50, 50,90,20);
        f.add(cb);
        f.setLayout(null);
        f.setSize(400,300);
        JLabel label1 = new JLabel();
         f.add(label1);
```

JComboBox – Ví dụ 2


```
        label1.setSize(300, 200);
        cb.addActionListener(new ActionListener(){
            @Override
            public void actionPerformed(ActionEvent e) {
                label1.setText("Bạn vừa chọn : " +
cb.getSelectedItem());        }
        });
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);        }
    public static void main(String[] args) {
        new JComboExample();
    }
}
```

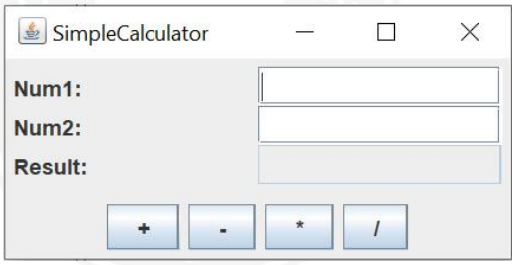

Bài tập

1) Viết chương trình đếm số lần click button.



Bài tập

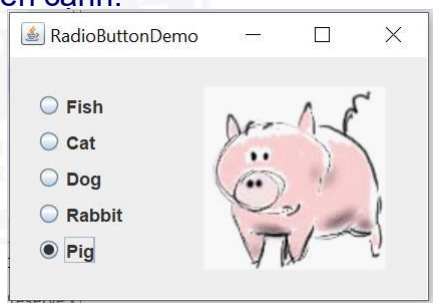
2) Viết chương trình máy tính để thực hiện các phép cộng, trừ, nhân chia. Khi click chọn phép tính, nếu chưa nhập số hoặc số không hợp lệ sẽ hiện thông báo dạng dialog.



Bài tập

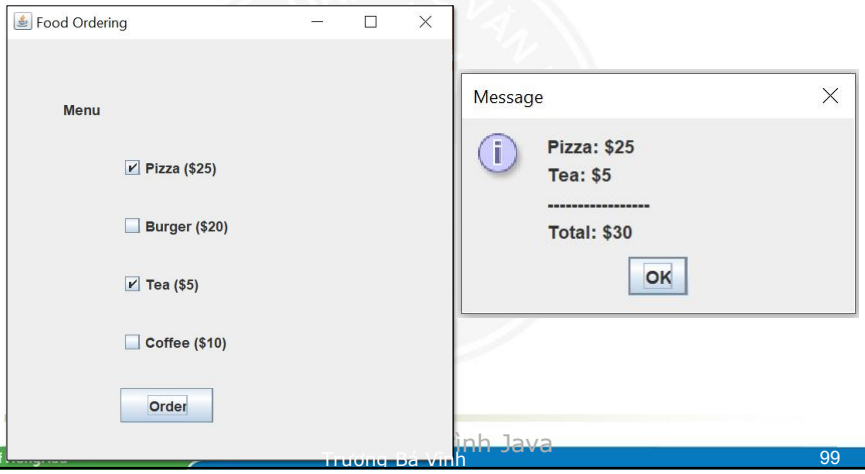
3) Tạo 1 frame, trong đó có 1 combobox có các lựa chọn: White, Green, Blue, Yellow, Red. Khi chọn màu nào thì màu nền của frame sẽ đổi thành màu tương ứng.

4) Viết ứng dụng như sau. Khi chọn con vật nào thì ảnh của con vật đó sẽ hiển thị bên cạnh.



Bài tập

5) Viết ứng dụng đặt thức ăn như sau. Khi click nút Order sẽ hiện dialog chứa thông tin các món đã đặt và số tiền.



JTable

- Là đối tượng được dùng để hiển thị dữ liệu dạng bảng, bao gồm các hàng và các cột.
- Một số constructor:
  - **JTable():** tạo ra 1 bảng gồm các ô trống.
  - **JTable(Object[][] rows, Object[] columns):** tạo 1 bảng với dữ liệu ban đầu

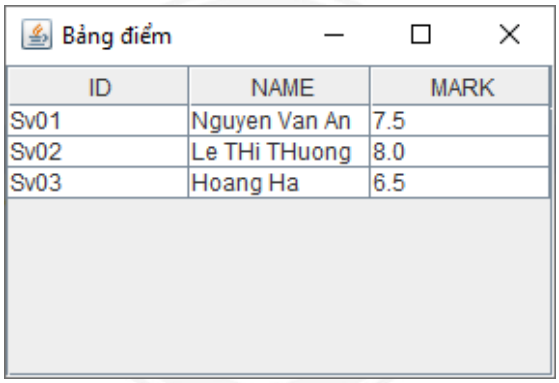
JTable – Một số phương thức thường dùng

- **setModel(TableModel model):** định nghĩa danh sách dữ liệu hiển thị trong JTable.
- **setColumnModel(TableColumnModel model):** định nghĩa tên hiển thị của các cột trong bảng.
- **isCellSelected(int r, int c):** trả về true khi ô dữ liệu ở vị trí (r,c) đang được chọn, ngược lại trả về false.
- **isColumnSelected(int c):** trả về true khi ô dữ liệu ở cột c đang được chọn, ngược lại trả về false.
- **isRowSelected(int c):** trả về true khi ô dữ liệu ở hàng c đang được chọn, ngược lại trả về false.
- **int getSelectedRow():** trả về chỉ số của dòng đang được chọn

JTable – Một số phương thức thường dùng

- **getValueAt(int r, int c):** trả về giá trị dữ liệu tại vị trí (r,c).
- **getRowCount():** trả về số dòng trong bảng.
- **getColumnCount():** trả về số cột trong bảng.

JTable – Ví dụ 1



JTable – Ví dụ 1

```
import javax.swing.*;
public class TableExample {
    JFrame f;
    TableExample(){
        f=new JFrame("Bảng điểm");
        String data[][]={ {"Sv01","Nguyen Van An","7.5"},
                {"Sv02","Le THi THuong","8.0"},
                {"Sv03","Hoang Ha","6.5"}};
        String column[]={"ID","NAME","MARK"};
        JTable jt=new JTable(data,column);
        jt.setBounds(30,40,200,300);
```

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

JTable – Ví dụ 1

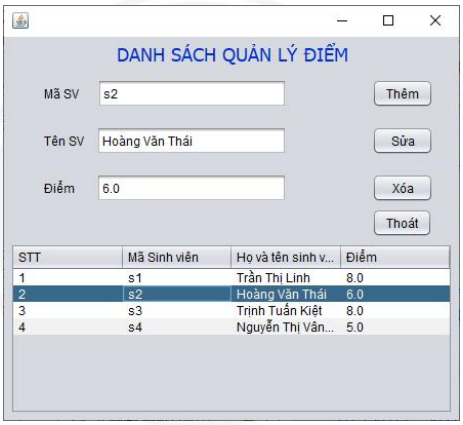
HOTLINE: 18
www.vhu

```
        JScrollPane sp=new JScrollPane(jt);
        f.add(sp);
        f.setSize(300,400);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
    public static void main(String[] args) {
        new TableExample();
    }
}
```


TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

JTable – Ví dụ 2

HOTLINE: 18
www.vhu



JTable – Ví dụ 2


```
class SinhVien {
    String ID, HoTen;
    double Diem;
    public SinhVien(String ID, String HoTen, double Diem) {
        this.ID = ID;
        this.HoTen = HoTen;
        this.Diem = Diem;
    }
    public String getID(){
        return ID;  }
    public String getHoTen(){
        return HoTen; }
    public double getDiem(){
        return Diem; }
}
```


JTable – Ví dụ 2


```
public class JTableDemo extends JFrame {
    Vector<SinhVien> studentList = new Vector<SinhVien>();
    DefaultTableModel model = new DefaultTableModel();
    JTable jTable1 = new JTable();
    public JTableDemo () {
        initComponents();
        jTable1.setModel(model);
        model.addColumn("STT");
        model.addColumn("Mã sinh viên");
        model.addColumn("Họ và tên sinh viên");
        model.addColumn("Điểm");
        hienThiList();
    }
```

JTable – Ví dụ 2


```
private void hienThiList(){
    model.setRowCount(0);    // xoa du lieu trong model
    int i=0;
    while (i<=studentList.size()-1){
        SinhVien s=studentList.get(i);
        model.addRow(new Object []
            {++i,s.getID(),s.getHoTen(),s.getDiem()});
    }
}
private void initComponents () {
    …..
    jTable1.addMouseListener(new MouseAdapter() {
        public void mouseClicked(MouseEvent evt) {
            jTable1MouseClicked(evt); }});
```


JTable – Ví dụ 2

```
jButtonAdd.addActionListener(new ActionListener() { public
    void actionPerformed(ActionEvent evt) {
        btnThemActionPerformed(evt); }});
jButtonEdit.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        btnSuaActionPerformed(evt); }});
jButtonDelete.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        btnXoaActionPerformed(evt); }});
jButtonExit.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
    …..
        btnThoatActionPerformed(evt); }});
```

JTable – Ví dụ 2

```
private void btnThemActionPerformed(ActionEvent evt) {
    studentList.add(new SinhVien(tfMaSV.getText(),
            tfTenSV.getText(),
            Double.parseDouble(tfDiem.getText())));
    int i=studentList.size()-1;
    SinhVien s=studentList.get(i);
    model.addRow(new Object []{
        ++i,s.getID(),s.getHoTen(),s.getDiem()});
}
```


TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

JTable – Ví dụ 2

```
private void btnSuaActionPerformed (ActionEvent evt) {
  int i=jTable1.getSelectedRow();
  if (i>=0){
        model.setValueAt(tfMaSV.getText(), i, 1);
        model.setValueAt(tfTenSV.getText(), i, 2);
        model.setValueAt(tfDiem.getText(), i, 3);
    }
}
```

JTable – Ví dụ 2

```
private void btnXoaActionPerformed (ActionEvent evt)   {
        int i=jTable1.getSelectedRow();
        if(i>=0){
          model.removeRow(i);
    }
}
private void btnThoatActionPerformed (ActionEvent evt) {
      System.exit(0);
}
```

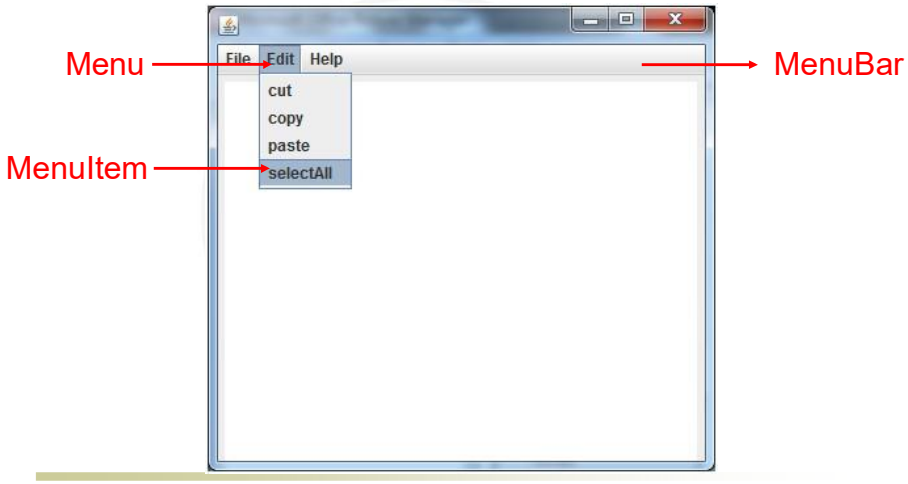

JTable – Ví dụ 2

```
private void jTable1MouseClicked (MouseEvent evt) {
      int i=jTable1.getSelectedRow();
      if (i>=0){
      tfMaSV.setText(model.getValueAt(i, 1).toString());
      tfTenSV.setText(model.getValueAt(i, 2).toString());
      tfDiem.setText(model.getValueAt(i, 3).toString());
      }
}
public static void main(String args[]) {
      new JTableDemo(); }
}
```

Menu

- Một hệ thống menu cung cấp một thanh menu (**Menu Bar**) bên dưới thanh tiêu đề của cửa sổ.
- Thanh menu này chứa nhiều item và được gọi là **Menu**.
- Khi chúng ta click vào một menu, nó sẽ hiện ra và hiển thị các lựa chọn khác nhau và được gọi là **Menu Item.**

Menu



JMenuBar

- JMenuBar là một class và được sử dụng để tạo một thanh trình đơn (menubar).
- Constructor:

  JMenuBar(): Tạo một thanh trình đơn mới

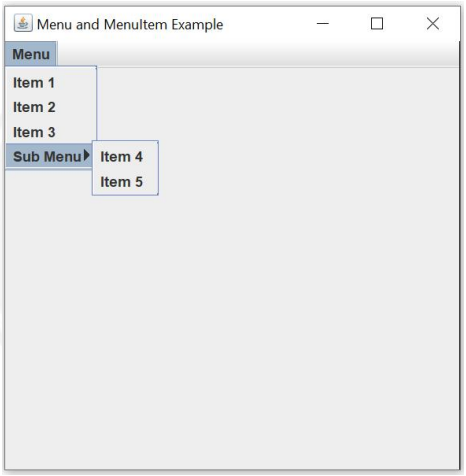
JMenu

- JMenu là một class và được sử dụng để tạo một Menu.
- Constructors
  - **JMenu():** Tạo một thể hiện JMenu không có text.
  - **JMenu(String s):** Tạo một thể hiện JMenu có text xác định.

JMenuItem

- JMenuItem là một class và được sử dụng để tạo một MenuItem.
- Constructors:
  - **JMenuItem():** tạo một thể hiện JMenuItem không có hình ảnh và text trống.
  - **JMenuItem (String s):** tạo một thể hiện JMenuItem với text cụ thể.
  - **JMenuItem (Icon i):** tạo một thể hiện JMenuItem với 1 ảnh xác định.
  - **JMenuItem (String s, Icon i):** tạo một thể hiện JMenuItemvới text xác định có ảnh.

Menu – Ví dụ 1



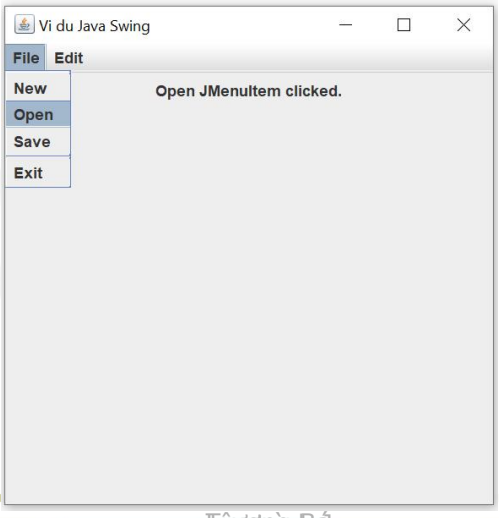
Menu – Ví dụ 1

```
import javax.swing.*; class
MenuExample {
        JMenu menu, submenu;
        JMenuItem i1, i2, i3, i4, i5;
        MenuExample(){
        JFrame f= new JFrame("Menu and MenuItem Example");
        JMenuBar mb=new JMenuBar();
        menu=new JMenu("Menu");
        submenu=new JMenu("Sub Menu");
        i1=new JMenuItem("Item 1");
        i2=new JMenuItem("Item 2");
        i3=new JMenuItem("Item 3");
```


Menu – Ví dụ 1

```
        i4=new JMenuItem("Item 4");
        i5=new JMenuItem("Item 5");
        menu.add(i1); menu.add(i2); menu.add(i3);
        submenu.add(i4); submenu.add(i5);
        menu.add(submenu);
        mb.add(menu);
        f.setJMenuBar(mb);      f.setSize(400,400);
        f.setLayout(null);      f.setVisible(true);
}
public static void main(String args[])
{
            new MenuExample();
}}
```

Menu – Ví dụ 2



Menu – Ví dụ 2

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class SwingMenuDemo implements ActionListener {
  private JFrame mainFrame;
  private JLabel statusLabel;
  public SwingMenuDemo(){
    prepareGUI();
    showMenuDemo();
  }
  private void prepareGUI(){
    mainFrame = new JFrame("Vi du Java Swing");
    mainFrame.setSize(400,400);
    mainFrame.setLayout(new FlowLayout());
```

Menu – Ví dụ 2

```
    statusLabel = new JLabel("",JLabel.CENTER);
    statusLabel.setSize(350,100);
    mainFrame.add(statusLabel);
}
private void showMenuDemo(){
    //Tao mot menu bar
    JMenuBar menuBar = new JMenuBar();
    //Tao cac menu
    JMenu fileMenu = new JMenu("File");
    JMenu editMenu = new JMenu("Edit");
    //Tao cac item
    JMenuItem newMenuItem = new JMenuItem("New");
    JMenuItem openMenuItem = new JMenuItem("Open");
```


Menu – Ví dụ 2

```
    JMenuItem saveMenuItem = new JMenuItem("Save");
    JMenuItem exitMenuItem = new JMenuItem("Exit");
    JMenuItem cutMenuItem = new JMenuItem("Cut");
    JMenuItem copyMenuItem = new JMenuItem("Copy");
    JMenuItem pasteMenuItem = new JMenuItem("Paste");
    newMenuItem.addActionListener(this);
    openMenuItem.addActionListener(this);
    saveMenuItem.addActionListener(this);
    exitMenuItem.addActionListener(this);
    cutMenuItem.addActionListener(this);
    copyMenuItem.addActionListener(this);
    pasteMenuItem.addActionListener(this);
```

Menu – Ví dụ 2

```
//Them item vao cac menu
fileMenu.add(newMenuItem);
fileMenu.add(openMenuItem);
fileMenu.add(saveMenuItem);
fileMenu.addSeparator();
fileMenu.add(exitMenuItem);
editMenu.add(cutMenuItem);
editMenu.add(copyMenuItem);
editMenu.add(pasteMenuItem);
//Them menu vao menubar
menuBar.add(fileMenu);
menuBar.add(editMenu);
```


127

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh
HOTLINE: 18
www.vhu

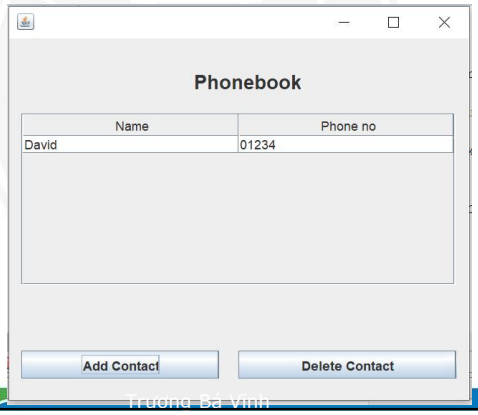
Menu – Ví dụ 2

```
//Them menubar vao frame
mainFrame.setJMenuBar(menuBar);
mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
mainFrame.setVisible(true);
}
public void actionPerformed(ActionEvent e) {
    statusLabel.setText(e.getActionCommand()
    + " JMenuItem clicked.");
}
public static void main(String[] args){
    SwingMenuDemo swingMenuDemo = new SwingMenuDemo();
}
}
```


128

Bài tập

1) Viết ứng dụng sau (khi click Add Contact sẽ hiện lần lượt 2 Input dialog để nhập Tên và SĐT)



Bài tập

2) Xây dựng 1 frame có 1 label với nội dung “Hello Java!” và thanh menu gồm 2 menu: Màu chữ và Màu nền. Trong mỗi Menu có 3 MenuItem: Xanh, Vàng, Đỏ. Khi chọn 1 MenuItem sẽ thay đổi màu chữ của label và màu nền của frame tương ứng.